

# Neuron Architecture

---

Hybrid execution model for the workflow engine: plans are generated and orchestrated server-side, but task execution can be routed to external processes ("neurons") — including desktop clients holding user-specific MCP connections — or to in-cluster workers, based on declared capabilities and scope.

---

## Motivation

---

The baseline architecture (see `ARCHITECTURE.md`) executes all tasks in server-side worker processes. This works for generic compute (scripts, HTTP, LLM calls) but breaks for user-specific integrations:

- **MCP connections are per-user.** A desktop client authenticated to Google Drive as user `abc` cannot legally (or safely) process a Drive task on behalf of user `xyz`.
- **Capability availability varies by client.** A desktop binary ships with different MCP servers than another; a web client has none.
- **Trust boundaries differ.** A server-side worker has database credentials and service-account trust; a user-installed desktop neuron does not.

The neuron architecture introduces a routing layer that matches tasks to executors by capability, scope, and authorization, while keeping plan generation and orchestration centralized.

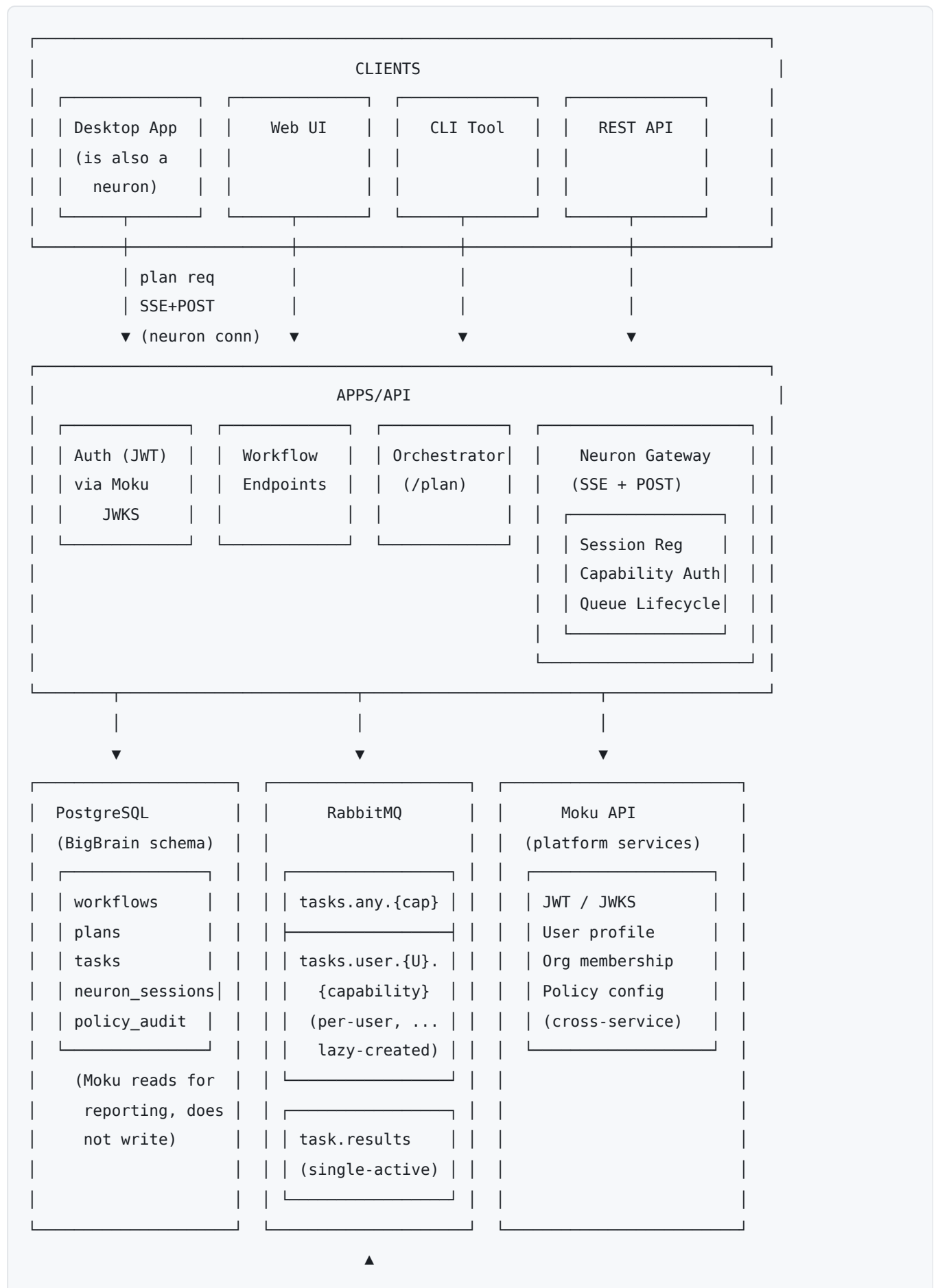
---

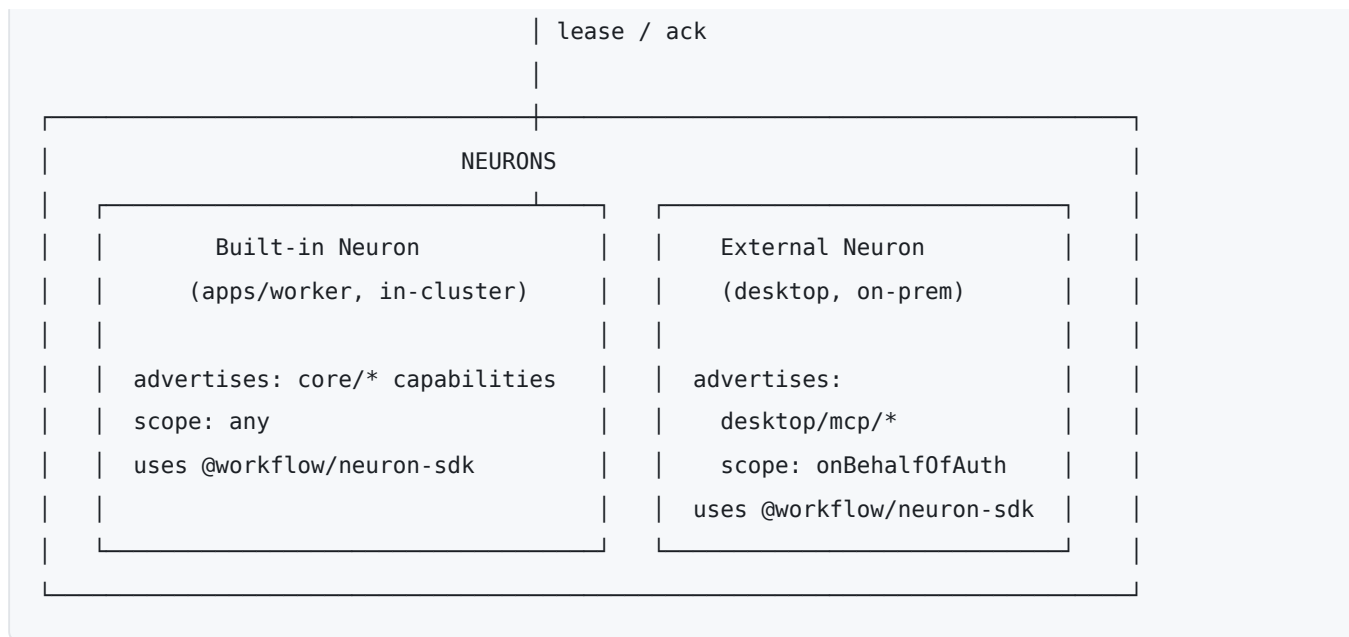
## Core Concepts

---

- **Neuron** — any process that executes tasks. Identified by an authenticated identity (JWT from Moku), advertises one or more capabilities at registration time. The built-in `apps/worker` is itself a neuron; external neurons (desktop, per-tenant agents) connect over the wire protocol.
  - **Capability** — a typed function signature that a neuron can execute. Identified by a namespaced type string ( `core/script.python` , `desktop/mcp/drive.read` ). Schemas for input/output are not global; they travel with the plan (see *Plan-Time vs Registration-Time Schemas*).
  - **Scope** — who a capability advertisement is valid for. Either `any` (any user's task) or `{ onBehalfOf: userId }` (only that user's tasks). Scope is enforced by the gateway at both registration time and routing time.
  - **Gateway** — the server-side component that accepts neuron connections, authorizes capability advertisements, binds consumer channels to per-user queues, and forwards work. Mounted as routes on `apps/api` for v1, isolated enough to extract into its own service if scale demands.
  - **On-behalf-of** — the user identity a task executes for. Set at plan time (from the workflow's owner), stamped on every task, used to select the target queue.
-

# System Diagram





## Execution Model

### Registration

A neuron opens an SSE stream to `/neuron/events` (server → neuron) and sends a `register` frame via `POST /neuron/register` with its JWT in the `Authorization` header. The frame carries:

- `neuronId` — stable id per device/process
- `capabilities[]` — each with `type`, `scope`, `concurrency`

The gateway:

1. Verifies the JWT via cached Moku JWKS.
2. For each advertised capability, checks the **namespace authorization table** — which identities are permitted to advertise which namespace prefixes.
3. Invokes `PolicyEngine.checkRegistration()` (see *Policy Integration*).
4. For user-scoped advertisements, verifies the auth token is authorized for that user.
5. Lazily declares the matching RabbitMQ queues and opens consumer channels with `prefetch = concurrency`.
6. Persists the session to `neuron_sessions` for UI visibility and routing lookups.

Registration is the **routing concern** only. No schemas are exchanged at registration time — schemas live with each plan.

### Capability Scoping

Scope is a property of each capability advertisement, not the neuron as a whole. One neuron may advertise mixed-scope capabilities:

```
{
  "capabilities": [
    { "type": "desktop/mcp/drive.read", "scope": { "onBehalfOf": "user:abc" }, "concurrency": 1 },
    { "type": "desktop/mcp/slack.post", "scope": { "onBehalfOf": "user:abc" }, "concurrency": 1 },
    { "type": "core/script.python", "scope": "any", "concurrency": 1 }
  ]
}
```

The auth token's user binding is the **only** authority for `onBehalfOf` scope — a neuron cannot advertise a scope it is not authorized for; registration is rejected.

## Per-User Queues

Queues for user-scoped capabilities are **lazily declared per (user, capability)** pair:

- Name: `tasks.user.{userId}.{capabilityType}` (e.g., `tasks.user.abc.desktop.mcp.drive.read`)
- `durable: true`
- `x-expires: 7d` — broker auto-deletes queue after 7d with no consumers and no messages
- `x-message-ttl: 24h` — individual tasks expire if unclaimed past this window
- DLX → `tasks.expired` — exposes timed-out tasks to the planner for re-plan or fail decisions

Queues for scope- `any` capabilities use a shared queue per capability type: `tasks.any.{capabilityType}`.

## Leasing and Restart Semantics

Tasks are leased, not dequeued. A neuron consumes a message without acking; the gateway writes the task's DB row to `assigned` with a lease expiry. The ack is sent only on terminal success. On disconnect, the broker redelivers the message automatically; a reaper flips stale `assigned` rows back to `pending` and the task resumes on the next consumer (possibly the same user's different device).

**Consequence for handler authors:** handlers must be idempotent, or use capability-native idempotency keys (e.g., Google Drive `requestId`). Restart semantics mean "a task that completed a side effect but failed to ack will re-run." This is a per-handler contract, not a framework-level guarantee.

## Concurrency

Each capability advertisement carries `concurrency`. The gateway translates this into AMQP `basic.qos prefetch_count` on the consumer channel, so the broker handles backpressure — once the neuron has N unacked messages, it stops receiving new ones until acks flow.

Neurons may send `UPDATE_CAPABILITY` frames to change concurrency mid-session (e.g., desktop plugged in); the gateway applies new prefetch on next delivery without disturbing in-flight leases.

## Heartbeats

Every ~10s, the neuron sends:

```
{ "type": "heartbeat", "neuronId": "...", "inFlight": [
  { "leaseId": "...", "taskId": "...", "startedAt": "..." }, ...
]}
```

Heartbeats (a) renew all active lease expiries in one round-trip, (b) let the gateway detect wedged neurons (inFlight full + no progress), (c) power the UI's "3/4 busy" display.

## Plan-Time vs Registration-Time Schemas

Capabilities are named routing addresses; **their schemas are not globally registered**. Instead:

- The client initiating a workflow owns the handler catalog — the set of capability types available in its context, each with input/output schemas (zod → JSON Schema). The desktop knows its MCP tools; the server knows its bundled handlers.
- At plan creation, the client posts the catalog alongside the goal:

```
POST /plan
{
  "goal": "summarize the sales deck in Drive and post to #launches",
  "onBehalfOf": "user:abc",
  "handlers": [
    { "type": "desktop/mcp/drive.read", "input": {...}, "output": {...} },
    { "type": "core/llm.summarize", "input": {...}, "output": {...} },
    { "type": "desktop/mcp/slack.post", "input": {...}, "output": {...} }
  ]
}
```

- The LLM planner receives the catalog in its prompt context and emits a typed plan (dot-path bindings from context into each step's input).
- The planner validates its own plan against the catalog before persisting (type-compatibility check on every binding).
- The catalog is **snapshotted with the plan** — replay, debugging, re-execution use the schemas that were valid at plan time, not whatever is current.

Execution-time validation happens in the neuron SDK (input-schema check before invoking the handler, output-schema check before acking). Schema mismatches → specific nack codes; the orchestrator surfaces them to the user with a clear "plan expected shape X, neuron produced shape Y" error.

## Capability Namespacing and Authorization

Capability types are namespaced: `{owner}/{package}/{name}`. Namespaces are not cosmetic — they are the primary key on the **namespace authorization table** that governs which identities are permitted to advertise which capabilities.

| Namespace       | Permitted advertisers                       | Typical scope                |
|-----------------|---------------------------------------------|------------------------------|
| core/*          | Server service identities (built-in worker) | any                          |
| desktop/*       | Signed/attested desktop binary identities   | onBehalfOfAuth               |
| org/{orgId}/*   | Tenant-installed extensions                 | any within org               |
| user/{userId}/* | Personal extensions                         | onBehalfOfAuth for that user |

Without enforced namespace authorization, capability collision leads to silent mis-routing or capability squatting (a malicious neuron claiming `core/*` addresses and siphoning tasks). The gateway rejects any registration where the advertiser's identity is not authorized for the namespace — collisions become visible failures at registration, not silent routing failures at runtime.

## Wire Protocol

Transport: **SSE for server** → **neuron + HTTPS POST for neuron** → **server**, with correlation ids linking responses. Enterprise-friendly (traverses proxies cleanly, looks like normal HTTPS), matches the transport style of OpenAI/Anthropic streaming APIs and MCP's HTTP surface.

### Message Types (overview — full schema in `packages/neuron-protocol`)

#### Neuron → Server (POST):

- `register` — open session with capabilities
- `update-capability` — change concurrency or scope for an existing capability
- `heartbeat` — lease renewal + in-flight state
- `ack` — task completed successfully, with output
- `nack` — task failed (retryable | terminal) with reason
- `progress` — in-progress status update for UI

#### Server → Neuron (SSE):

- `registered` — registration confirmed, includes effective capability bindings after policy/auth filters
- `lease` — a task to execute ( `leaseId` , task payload with input schema)
- `cancel` — cancel a specific lease (triggered by workflow cancellation)
- `capability-revoked` — a capability was deauthorized mid-session; in-flight leases on that capability must fail
- `disconnect` — graceful close from server

SSE resumption uses `Last-Event-ID` for short reconnects; longer disconnects go through a fresh `register` (all prior leases already redelivered by the broker).

## Policy Integration

---

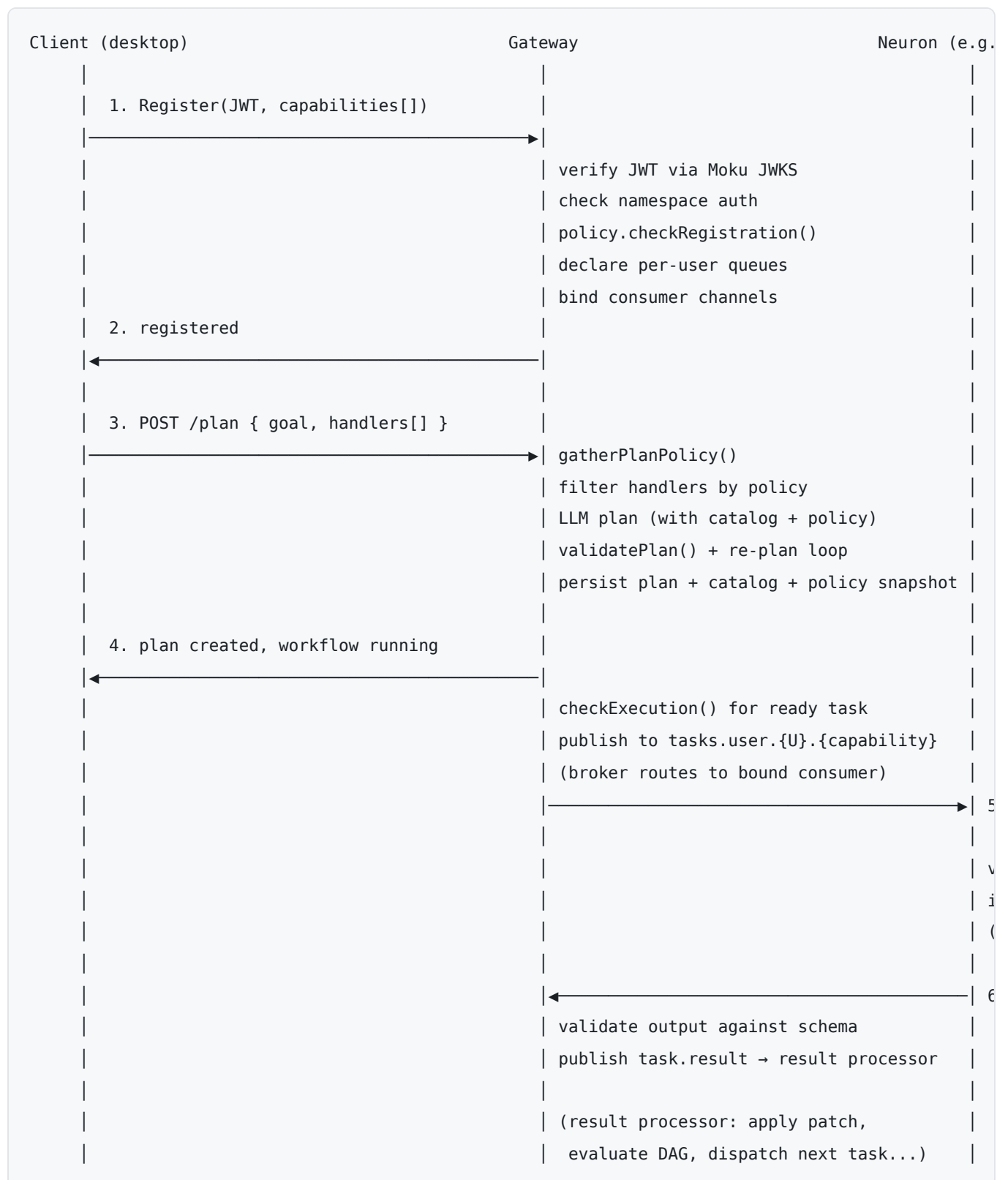
Four hook points, each at a distinct lifecycle stage. Initially implemented as a `NoOpPolicyEngine` that always allows; real engine design deferred. Interface shipped in `packages/policy` :

| Hook                           | Call Site                              | Input                       | Failure Mode                                                  |
|--------------------------------|----------------------------------------|-----------------------------|---------------------------------------------------------------|
| <code>checkRegistration</code> | Gateway, in register handler           | identity, capability, scope | Registration rejected with code                               |
| <code>gatherPlanPolicy</code>  | Planner, start of plan pipeline        | goal, onBehalfOf, handlers  | Opaque <code>PolicyContext</code> returned (NoOp → undefined) |
| <code>validatePlan</code>      | Orchestrator, after LLM plan emission  | plan, policyContext         | Re-plan with violations, bounded retries, else terminal fail  |
| <code>checkExecution</code>    | Dispatcher, immediately before publish | task, plan, policyContext   | Task fails with "policy changed since plan creation"          |

The `PolicyContext` returned by `gatherPlanPolicy` is **persisted alongside the plan**. `checkExecution` uses it as the at-plan-time snapshot for divergence detection — errors can name the specific policy that changed between plan time and execution time.

---

## Data Flow: Hybrid Plan Execution





# Moku Boundary (Data Ownership)

---

BigBrain owns its own Postgres **schema** (not a separate physical database). Moku retains read access for reporting and is the schema authority (migrations reviewed through Moku's coordination process); no other service writes to BigBrain's tables. This solves the cross-service coordination problem at the source.

**BigBrain ↔ Moku API consumption surface:**

| Capability                    | Cadence               | Cached               |
|-------------------------------|-----------------------|----------------------|
| JWT verification (JWKS fetch) | Rare (key rotation)   | Yes, in-memory       |
| User profile lookup           | On-demand (UI, audit) | Yes, short TTL       |
| Org membership                | On auth               | Yes, per-session     |
| Policy config                 | On plan start         | Yes, by version hash |

Hot-path operations — task state transitions, lease renewals, heartbeats, context patches — hit the BigBrain schema directly. Moku is not in the hot path.

---

## Key Design Decisions

| Decision                | Choice                                                                                                              | Rationale                                                                              |
|-------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Execution model         | Hybrid neuron-based routing by capability                                                                           | Enables user-specific MCP execution (desktop) alongside server-side compute            |
| Gateway placement       | Mounted on <code>apps/api</code> for v1, isolated enough to extract                                                 | Faster to ship; scale-out path preserved                                               |
| Transport               | SSE (server → neuron) + HTTPS POST (neuron → server)                                                                | Enterprise-network friendly; WebSocket fails behind many corporate proxies             |
| Auth                    | JWT from Moku (existing IDP integration); verified in-process via cached JWKS                                       | Reuses existing auth; no per-request Moku round-trip                                   |
| Capability scoping      | Per-advertisement ( <code>any</code> or <code>onBehalfOf:userId</code> ); auth token bounds <code>onBehalfOf</code> | Prevents user-scope spoofing; one neuron may mix scopes                                |
| Queue topology          | Per-user queues lazily created, broker-reaped via <code>x-expires</code>                                            | Natural per-user ordering; pending-state visibility; bounded growth                    |
| Lease semantics         | Unacked AMQP messages + DB <code>assigned</code> state + lease expiry reaper                                        | Restart-from-scratch on disconnect; no partial-progress preservation                   |
| Handler idempotency     | Author contract (framework does not provide mid-task checkpointing)                                                 | Simpler framework; leverages capability-native idempotency keys                        |
| Concurrency             | Per-capability advertisement → <code>basic.qos prefetch_count</code>                                                | Broker handles backpressure; no cross-capability global cap in v1                      |
| Schema ownership        | Per-plan catalog, snapshotted with plan; no global capability registry                                              | Enables desktop-unique capabilities; avoids coordination bottleneck                    |
| Namespace authorization | Capability type = <code>{owner}/{package}/{name}</code> ; gateway enforces identity → namespace table               | Prevents capability squatting; silent mis-routing becomes visible registration failure |
| Wire protocol           | <code>@workflow/neuron-protocol</code> package shared between gateway and SDK                                       | Single source of truth for message shapes                                              |
| Neuron SDK              | <code>@workflow/neuron-sdk</code> ; used by built-in worker too                                                     | One implementation, one test surface                                                   |
| Policy integration      | Four hooks (register / gather / validate / execute) with NoOp shim shipped first                                    | Locks call sites without blocking on real engine design                                |
| Moku boundary           | BigBrain owns its schema; Moku retains read for reporting                                                           | Coordination at schema-change time, not runtime                                        |

## Failure Modes

---

### Neuron disconnect mid-task

Broker redelivers unacked message. Reaper flips `assigned` → `pending`. Next consumer binding (same user, any device) re-leases. Handler must be idempotent.

### User-scoped neuron offline at dispatch

Task sits in per-user queue ( `pending` state in DB, visible in UI). `x-message-ttl` bounds the wait window; on expiry the task DLX's to the planner for re-plan or fail.

### Capability collision attempt

Gateway rejects registration at the namespace-authorization check. Visible failure, surfaced to neuron with a specific error code.

### Schema mismatch at execution

Neuron SDK pre-validates input before invoking handler; rejects with `schema-mismatch` nack. Output validated on ack; a mismatching output also nacks. Either surfaces through the result processor as a terminal task failure with a structured error.

### Policy change mid-workflow

`checkExecution` compares at-plan-time snapshot to current policy. Divergence → task fails with a specific `policy-changed` error that names the rule and the plan-time version. The workflow can be re-planned against current policy if the user retriggers.

### Gateway restart

Neurons notice SSE disconnect, attempt `register` via reconnection loop. All in-flight leases were unacked → broker redelivered them → reaper already moved them back to pending. Nothing to recover explicitly; connection re-establishment picks up naturally.

---

## Open Items (Deferred)

---

- **Real policy engine design** — interface stable; engine design (rule language, classification propagation, obligations) deferred.
- **Device credentials** — v1 uses user-session JWTs; device-specific credentials with per-device revocation are a follow-up.
- **Cross-capability global concurrency cap** — per-capability caps only in v1; add global if a real case emerges.
- **Browser-based neurons** — SDK is node-first; wire protocol stays browser-compatible for future.
- **Per-capability feature tags for version drift** — not in v1; add when needed.